

DELIMITER \$\$

```
CREATE PROCEDURE `GenerateDailySchedule`(IN inputUserID INT)
BEGIN
    DECLARE done          INT DEFAULT FALSE;
    DECLARE t_id          INT;
    DECLARE t_time        INT;
    DECLARE c_id          VARCHAR(50);
    DECLARE c_user        INT;
    DECLARE c_start       TIME;
    DECLARE c_end         TIME;
    DECLARE currentStart  TIME DEFAULT '08:00:00';
    DECLARE existingID    INT DEFAULT NULL;

    DECLARE task_cursor CURSOR FOR
        SELECT TaskID, EstimatedTime
        FROM Task
        WHERE UserID = inputUserID
        ORDER BY Priority DESC;

    DECLARE class_cursor CURSOR FOR
        SELECT uc.ClassID, uc.UserID, c.StartTime, c.EndTime
        FROM UserClasses AS uc
        JOIN Class c ON uc.ClassID = c.ClassID
        WHERE uc.UserID = inputUserID
        AND c.Days LIKE CONCAT('%',
            CASE DAYOFWEEK(CURDATE())
            WHEN 2 THEN 'M'
            WHEN 3 THEN 'T'
            WHEN 4 THEN 'W'
            WHEN 5 THEN 'R'
            WHEN 6 THEN 'F'
            END, '%')
        ORDER BY c.StartTime;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    SELECT ScheduleID INTO existingID
    FROM Schedule
    WHERE Date = CURDATE()
    LIMIT 1;

    SET done = FALSE;
```

```

IF existingID IS NULL THEN
    INSERT INTO Schedule (ScheduleID, Date)
    VALUES (
        (SELECT IFNULL(MAX(s.ScheduleID), 0) + 1 FROM (SELECT ScheduleID FROM
Schedule) AS s),
        CURDATE()
    );
    SET @newScheduleID = (SELECT MAX(ScheduleID) FROM Schedule);
ELSE
    SET @newScheduleID = existingID;
END IF;

DELETE FROM Blocks WHERE ScheduleID = @newScheduleID;

SET done = FALSE;
OPEN class_cursor;
class_loop: LOOP
    FETCH class_cursor INTO c_id, c_user, c_start, c_end;
    IF done THEN LEAVE class_loop; END IF;

    INSERT INTO Blocks (BlockID, Start, End, ScheduleID, TaskID, ClassID)
    VALUES (
        (SELECT IFNULL(MAX(b.BlockID), 0) + 1 FROM (SELECT BlockID FROM Blocks) AS
b),
        c_start,
        c_end,
        @newScheduleID,
        NULL,
        c_id
    );
END LOOP;
CLOSE class_cursor;

SET done = FALSE;
OPEN task_cursor;
read_loop: LOOP
    FETCH task_cursor INTO t_id, t_time;
    IF done THEN LEAVE read_loop; END IF;

    SELECT COUNT(*) INTO @conflict
    FROM Blocks
    WHERE ScheduleID = @newScheduleID
        AND currentStart < End
        AND ADDTIME(currentStart, SEC_TO_TIME(t_time * 60)) > Start;

```

```

SET done = FALSE;

WHILE @conflict > 0 DO
    SELECT MAX(End) INTO currentStart
    FROM Blocks
    WHERE ScheduleID = @newScheduleID
    AND currentStart < End
    AND ADDTIME(currentStart, SEC_TO_TIME(t_time * 60)) > Start;

    SET done = FALSE;

    SELECT COUNT(*) INTO @conflict
    FROM Blocks
    WHERE ScheduleID = @newScheduleID
    AND currentStart < End
    AND ADDTIME(currentStart, SEC_TO_TIME(t_time * 60)) > Start;

    SET done = FALSE;
END WHILE;

INSERT INTO Blocks (BlockID, Start, End, ScheduleID, TaskID, ClassID)
VALUES (
    (SELECT IFNULL(MAX(b.BlockID), 0) + 1 FROM (SELECT BlockID FROM Blocks) AS
b),
    currentStart,
    ADDTIME(currentStart, SEC_TO_TIME(t_time * 60)),
    @newScheduleID,
    t_id,
    NULL
);

SET currentStart = ADDTIME(currentStart, SEC_TO_TIME(t_time * 60));
END LOOP;
CLOSE task_cursor;

END$$

DELIMITER ;

```

DELIMITER //

```
CREATE TRIGGER AddTaskCheck
BEFORE INSERT ON Task
FOR EACH ROW
BEGIN
    IF new.Priority > 5 THEN
        SET new.Priority = 5;
    ELSEIF new.Priority < 1 THEN
        SET new.Priority = 1;
    END IF;
```

```
        IF new.EstimatedTime < 1 THEN
        SET new.EstimatedTime = 1;
        END IF;
    END //

DELIMITER ;
```